



Teacher Implementation Guide

AI + Coding Starter Kit | Grades 8-12

Implementation Guide

AI should help students code better, not avoid learning code.

AI should help students think better, not think less.

Standards summary: This resource may support CSF 9.2, CSF 13.1, CSF 15.1, CSF 16.1, CSF 16.2 when used as part of AI literacy, computer science, programming, digital ethics, or cybersecurity instruction. Detailed standards connection appears at the end of this document.

Implementation Purpose

This guide helps teachers introduce AI-supported coding in a practical, student-safe way. It can be used after an introductory AI literacy lesson or during a Python unit when students begin debugging code.

Implementation Goals

Teach students to use AI as a coding coach, not a code generator.

Build debugging habits: read, predict, test, revise, verify, and document.

Help students ask better coding questions.

Support academic integrity in AI-assisted programming.

Give teachers consistent language for acceptable and unacceptable AI support.

Recommended Sequence

When	Activity	Teacher Focus
Day 1	Lesson: AI as a Coding Coach	Coach vs. replacement; responsible coding prompts
Day 2	Ask AI Better Coding Questions handout	Prompt quality and student ownership
Day 3	Python Debugging Activity	Structured troubleshooting and verification
Day 4	Student-Safe AI Conversation Sample	Model what acceptable AI use looks like
Ongoing	Rubric used with coding assignments	Explain code, document AI support, verify results

Classroom Norms to Establish

AI use must follow teacher directions for each assignment.



Students should attempt to understand or trace code before asking AI for help.

Students should ask for hints, explanations, or debugging guidance before requesting fixes.

Students should not submit code they cannot explain.

Students should document how AI was used when required.

Students should not paste private information, passwords, API keys, or sensitive data into AI tools.

Suggested Teacher Language

In this class, AI can help you learn to code when I allow it. It can explain, hint, debug, and generate practice. It cannot replace your thinking, your testing, or your responsibility to understand the program you submit.

Administrator-Facing Value

Creates a practical academic integrity framework for AI-supported coding.

Connects AI literacy directly to Computer Science Foundations and programming instruction.

Supports students who get stuck while still requiring explanation and verification.

Builds career-connected habits: debugging, documentation, iteration, and ethical tool use.

Family Communication Option

Families can be told that students are learning how to use AI responsibly while coding. The goal is not to have AI write programs for students. The goal is to help students understand errors, practice debugging, ask better questions, and explain their own code.



Detailed Tennessee Standards Connection

Use this page as implementation guidance. Alignment depends on local district pacing, approved course placement, teacher directions, and how the resource is used in instruction.

Tennessee Standards Connection

This implementation guide helps teachers connect AI-supported coding routines to programming, debugging, ethical use, verification, and iterative software development.

Standards source: Tennessee Department of Education, Computer Science Foundations (C10H11), May 2023. Confirm final alignment against local district pacing, approved course placement, and teacher directions.

This resource may support the following Tennessee standards when used as part of AI literacy, computer science, programming, digital ethics, or cybersecurity instruction:

- CSF 9.2 - Troubleshooting Process: Students use a structured process to identify a problem, gather information, isolate causes, test a solution, verify the result, and document what they learned.
- CSF 13.1 - Social, Legal, and Ethical Issues: Students identify responsibilities related to ethical technology use, academic integrity, copyright, appropriate AI use, and responsible programming support.
- CSF 15.1 - Digital File Management: Students use organized file and folder practices when managing code files, assets, and project materials.
- CSF 16.1 - Programming Language: Students explore programming languages such as Python and explain how programmers use them to solve a variety of IT problems.
- CSF 16.2 - Software Development Life Cycle: Students connect planning, coding, testing, refinement, deployment, and maintenance to an iterative software-development process.